# Spam and Phishing Detection

CSIT375 AI and Cybersecurity

Dr Wei Zong

SCIT University of Wollongong

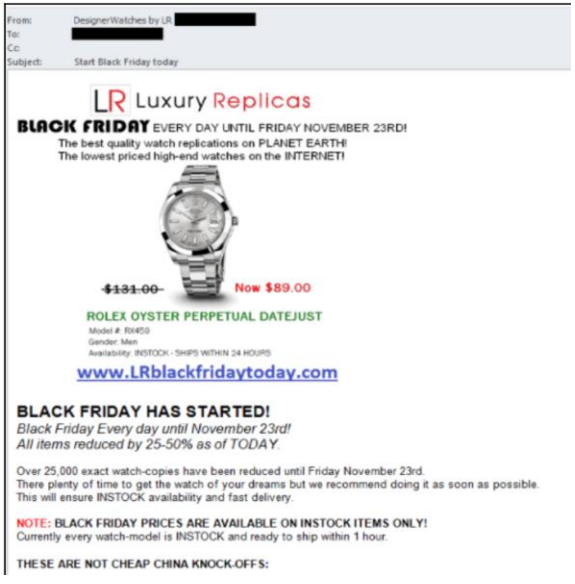
Disclaimer: The presentation materials come from various sources. For further information, check the references section

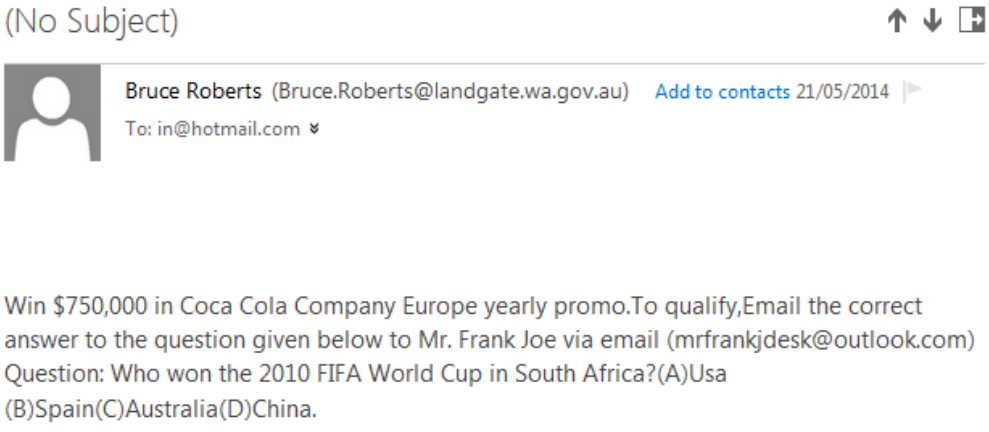
# Outline

- What are Spam and Phishing?
- Spam Filtering Techniques
- Pre-processing Text in Email Messages

# What is Spam

- Spam: Unsolicited bulk email or message
- Emails that involves sending identical or nearly identical messages to thousands (or millions) of recipients



# What are the problems?

- With a tiny investment, a spammer can send over 100,000 bulk emails per hour
- Waste users' time, may cause financial loss
  - Study shows: an average person spends 28% of the workday (≈ 670 hrs/year) reading and responding to emails
  - only 38% of the emails on an average are relevant and important
- Junk mails waste storage and transmission bandwidth
- Might include malware as executable files
- Spam is a problem because the cost is forced onto the recipient

# Statistics

- Spam accounts for 45% of all emails sent
- About 14.5 billion spam emails are sent every single day
- Spam costs $20.5 billion yearly (reduced network bandwidth, storage capacity)
- Spammers receive on average 1 click for every 12 million emails sent
  - Even with this response, spammers earn millions of dollars yearly
- 80% of all spam is sent by the same 100 spammers

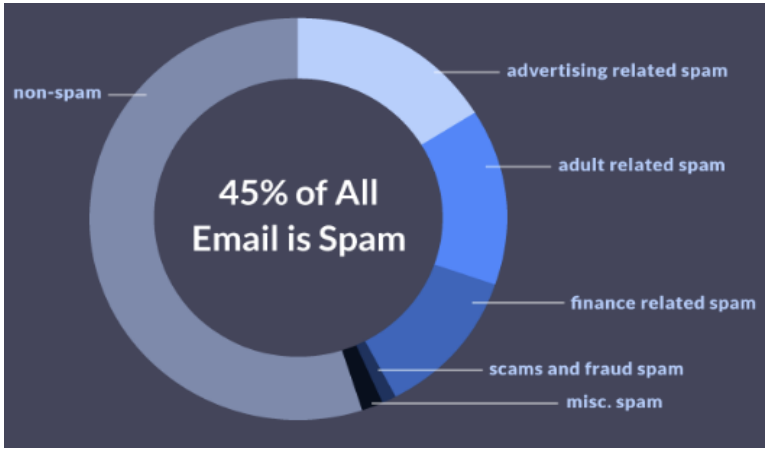


Image credit: https://www.propellercrm.com/blog/email-spam-statistics

# Phishing

- **Phishing:** attacks where a victim is lured to a fake web, and is deceived into disclosing personal data or credentials
- **Phishing URLs** seem like legitimate URLs, and redirect the users to phishing web pages, which mimic the look and feel of their target websites
  - ***URL (Uniform Resources Locator)*** is a web address that specifies the location of the webpage on a computer network
  - A typical URL http://www.example.com/index.html consists of several components:
    - Protocol type = http
    - Domain name = www.example.com
    - File name = index.html

# Phishing

- Phishing emails are a more serious threat than spam emails, because they aim to steal users' private information, such as bank accounts, passwords, SSNs

# Dropbox Phishing

- Attackers create fake sign-in pages for Dropbox as a part of credential harvesting
- They then use the stolen credentials to log in to legitimate sites and steal user data

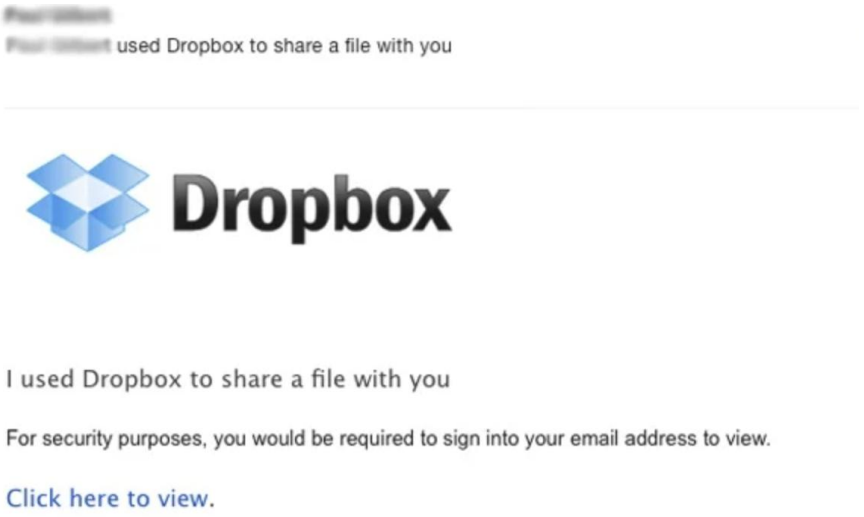





image credit: mailguard

# CEO Fraud

- **CEO fraud**: (also called whaling attack)
  - target: top executives of an organization
  - suffer from account takeovers due to stolen login credentials
- The scammer takes over CEO Account
- CEO's credentials are used to login
- Scam emails sent to all

# HOW CEO FRAUD IMPACTS YOU

## THE START

Attackers see if they can spoof your domain and impersonate the CEO (or other important people)

Bad guys often troll companies for months to gather the data necessary in pulling off a successful attack

## THE PHISH

Spoofed emails are sent to high-risk employees in the organization

To: Finance Department

Urgent wire transfer request! Please send $100,000 to new acct #987654-3210

To: CFO

Please pay this time-sensitive invoice. I'm on vacation and will be unavailable, no need to respond. - Your CEO

To: Human Resources

I need 5x $1000 physical Amazon gift cards. Please scratch them off and send me a clear picture of the barcodes.

## THE RESPONSE

Target receives email and acts without reflection or questioning the source

I better get this payment to the new account!

It's from the CEO, I'll take care of this for him!

Sounds important, I'll pick those up on my lunch break!

## THE DAMAGE

Social engineering was successful, giving hackers access to what they were after

Causing fraudulent wire transfers and massive data breaches

## THE RESULT

The fallout after a successful attack can be highly damaging for both the company and its employees

Resulting damage:

- Money is gone forever in most cases and only recovered 4% of the time
- CEO is fired
- CFO is fired
- Lawsuits are filed
- Intangibles - tarnished reputation, loss of trust, etc.
- Stock value drops

So... Think Before You Click!

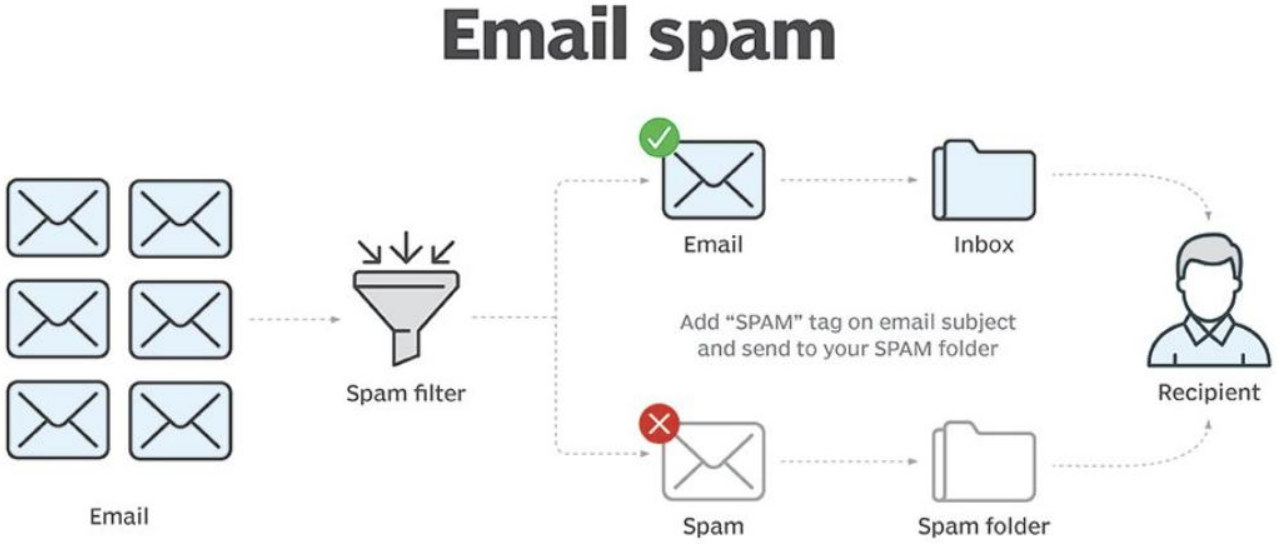
KnowBe4 Human error. Conquered.

© 2020 KnowBe4, Inc. All rights reserved.

SECURITY AWARENESS TRAINING | WWW.KNOWBE4.COM

# Spam Filtering

- Spam filter: Goal is to determine whether an incoming message is legitimate (i.e., non-spam, ham) or unsolicited (i.e., spam)



# Spam Filtering

- A typical data flow in spam filtering
- How input emails are processed and sent to the model
- To give data as input to the model, we'd need to transform them to some numerical format

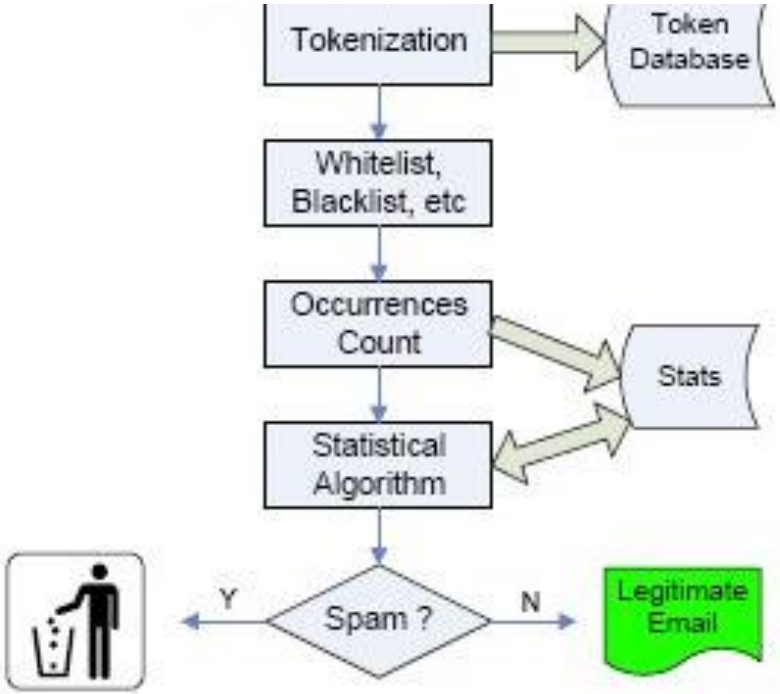





*Image credit: Figure from: Yan (2010) – Workload Characterization of Spam Email Filtering Systems*

# Spam Filtering

- ML-based spam classifiers were among the first applications of machine learning in the cyber security domain
  - Subsequently, they were among the first to be attacked
  - Attackers' goal is to modify spam emails (without changing the nature of the message) to bypass spam filters

- Recent spam filters increasingly rely on machine learning and neural networks approaches for email classification
  - These approaches are being extensively used by email service providers like Gmail, Outlook, or Yahoo

# Filtering Techniques

- Challenge-Response Filtering
- Blacklists and Whitelists
- Rule based filters
- Content based filters

# Challenge-Response Filtering

- When a sender sends an email message, a challenge (e.g., CAPTHCA) is sent to the sender. A legitimate user can solve the challenge easily
- But, if a spammer sends a large number of spam messages, it is difficult for the spammer to solve the challenge
- As a result, the challenge-response spam filtering system can differentiate legitimate emails from spam emails easily

- Limitation
  - Sometimes senders don’t or forget to reply to the challenge
  - Advance in ML techniques facilitates automatic challenge solving.

# Blacklists and Whitelists

- **Blacklists** of misbehaving servers or known spammers that are collected by several sites
- Sender id in the email is compared with the blacklist
- **Whitelists** are complementary to blacklists, and contain addresses of trusted contacts
- Use blacklists and whitelists for the first level filtering (before applying content checks) and not used as the only tool for making decision

- Limitation
  - Prone to wrong configurations with legitimate servers unable to exit from a list where they had been incorrectly inserted

# Rule based Filtering

- **Rule-based filtering techniques**
  - Apply static rules to discover similar patterns in a large number of spam and non-spam emails
  - Scores are assigned to each rule, and the scores are weighted based on the importance of the rule
  - If the total score > a predefined threshold, the message is labeled as spam
- Rules could be created based on:
  - Words and phrases, lots of uppercase characters, exclamation points, unusual Subject lines, special characters, web links, HTML messages, background colors, etc
- Limitation
  - Requires constant updating of the rules
  - Continually adapting strategies by spammers

# Content based Filtering

- Content based filtering techniques
  - The filter scans the content of incoming emails, looking for trigger keywords
    - E.g., keywords frequently used in spam emails, such as free, buy, application, mortgage
  - The content of the body and header of emails are scanned
- The frequency of occurrence and distribution of trigger words and phrases in the content of emails are used as features for training ML approaches, and afterward, for classifying new emails
  - Naïve Bayes classifiers are one of the early successful ML models for spam filtering
  - Other conventional ML approaches have been successfully applied, such as SVMs, k-nearest neighbors, decision trees, …
  - Deep neural networks are commonly used nowadays for spam classification

# Content based Filtering

- Scanning the body of emails explores the what in the email
  - Scanning the header of emails explores the who sent the email
- Email headers display important information, such as:
  - Message ID – an identifier generated by the sender's email service
    - There can be no two identical message IDs, hence, it helps to detect forged email headers
  - Sender address – is used to consult blacklists to check sender's domain reputation
  - DNS records – the DNS (Domain Name System) records of the sender allows to check the sender's SPF, DKIM, and DMARC policies regarding email authentication
    - SPF (Sender Policy Framework), DKIM (Domain Keys Identified Mail), DMARC (Domain-based Message Authentication, Reporting and Conformance)

# An example of a Gmail header

Original Message

| | |
|---|---|
| Message ID | <000000000000d7d44705c2854da2@google.com> |
| Created at: | Mon, May 17, 2021 at 2:57 PM (Delivered after 12 seconds) |
| From: | christian@gmail.com |
| To: | folderly@gmail.com |
| Subject: | Request to connect |
| SPF: | PASS with IP 209.85.220.69 Learn more |
| DKIM: | 'PASS' with domain belkins.io Learn more |
| DMARC: | 'PASS' Learn more |

# Spam filters in action - a Motivating Example

- How does an anti-spam algorithm behave in the classification of emails? - based on suspicious keywords, e.g. buy, shop
- Task: classify the email messages within a table, showing the number of occurrences of the individual keywords identified within the text of the emails, indicating the messages as spam or ham:

| Email | Buy | Shop | Spam or Ham? |
|---|---|---|---|
| 1 | 1 | 0 | H |
| 2 | 0 | 1 | H |
| 3 | 0 | 0 | H |
| 4 | 1 | 1 | S |

# Spam filters in action - a Motivating Example

- Need to assign a score to every single email message
  - This score will be calculated using a scoring function that takes into account the number of occurrences of suspicious keywords contained within the text
- A simple scoring function with weights $y = 2B + 3S$

| Email | B | S | 2B + 3S | Spam or Ham? |
|---|---|---|---|---|
| 1 | 1 | 0 | 2 | H |
| 2 | 0 | 1 | 3 | H |
| 3 | 0 | 0 | 0 | H |
| 4 | 1 | 1 | 5 | S |

- Threshold value to separate spam from ham: e.g. 4

# Pre-processing Text in Email Messages

- Before we can process documents (emails), it's important to convert these documents into numerical representations

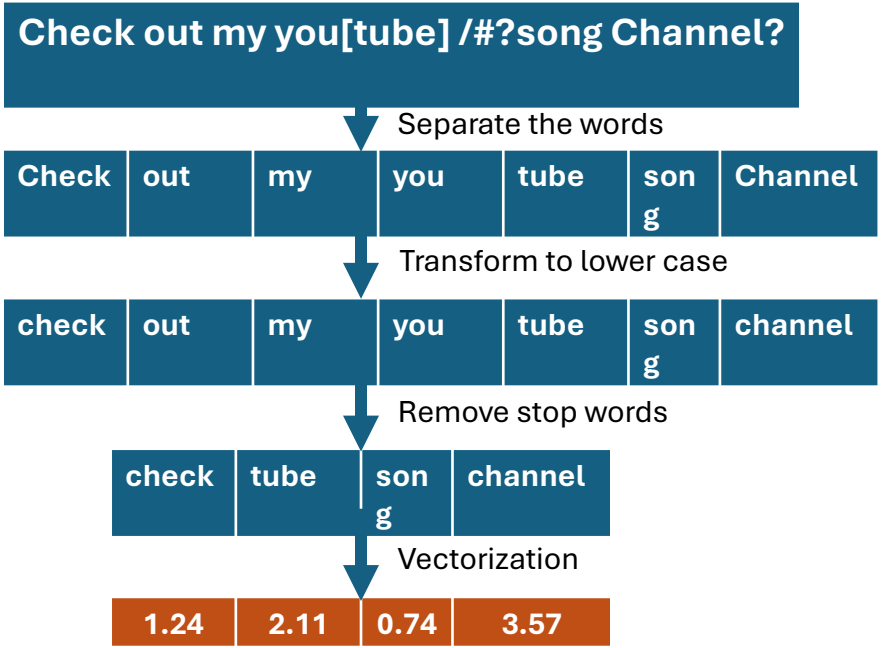
- Preprocessing text data in emails typically involves
  - Tokenization
    - Refers to separating the words in a text
    - Tokenization transforms an email into a sequence of representative symbols (tokens)
  - Vectorization
    - Transform to numerical format – ’vectors’
    - Text in Email Messages is preprocessed into numeric representation for use by ML models

# Tokenization

- Tokenization usually includes several steps.
- Remove punctuation signs (comma, period) or non-alphabetic characters (@, #, {, ])
- Remove stop words, such as for, the, is, to, some
  - These words appear in both spam and non-spam emails, and are not relevant for filtering
- Correct spelling errors or abbreviations
- Change all words to lower-case letters
  - I.e., the model should consider Text and text as the same word
- Stemming - transforming words to their base form
  - E.g., the words buy-bought or grill-grilled have a common root

# Tokenization

- Example of tokenization



# Vectorization

- We need to transform the tokens (terms) to numerical format
- Some methods:
  - Bag of words
  - n-grams
  - TF-IDF

```
tokenized_messages: {
    'A': ['hello', 'mr', 'bear'],
    'B': ['hello', 'hello', 'gunter'],
    'C': ['goodbye', 'mr', 'gunter']
}

# Bag-of-words feature vector column labels:
# ['hello', 'mr', 'doggy', 'bear', 'gunter', 'goodbye']
vectorized_messages: {
    'A': [1,1,0,1,0,0],
    'B': [2,0,0,0,1,0],
    'C': [0,1,0,0,1,1]
}
```

# Bag-of-Words

- ***Bag-of-words model***
  - The tokenized words in emails are represented as a bag (i.e., set) of words
- The term *bag* implies that the order of the words and the structure of the text is lost
- Each word is a token having a numeric feature representation
- Typically, the frequency of occurrence of each word is used as a feature for training a classifier
- Example
  - Text: John likes to watch movies. Mary likes movies too.
  - Bag-of-words listing the words and the frequency of each word:

    {"John":1,"likes":2,"to":1, "watch":1, "movies":2,"Mary":1,"too":1}

# Bag-of-Words

- Represent an email by the occurrence counts of each word
- What information is lost with this representation?
- **Ordering** of words is lost
  - Alice is quicker than Bob and Bob is quicker than Alice have the same vector representation
  - => n-grams
- **Term importance** is also lost
  - => TF-IDF

# *n*-Grams

- Instead of using single words as tokens, it is also possible to use *n* consecutive words, referred to as ***n-grams***
  - Combining several consecutive words together creates more specialized tokens
  - E.g., the word *play* is considered a neutral word, but the two-words phrase *play lotto* is less neutral
    - Such *n*-grams consisting of adjacent pairs of words are called bigrams
    - *n*-grams consisting of single words are called unigrams
- The *n*-grams model preserves the words order, potentially capture more information than the bag-of-words model

# TF-IDF

- Assume ***t*** distinct tokens remain after preprocessing
  - call them terms or the vocabulary
- Each token ***i***, in a document ***j***, is given a real-valued weight , $w_{ij}$
- Documents are expressed as ***t***-dimensional vectors

$$d_j = (w_{1j}, w_{2j}, \ldots, w_{tj})$$

# Document Collection

- A collection of *n* documents can be represented in the vector space model by a term-document matrix
- An entry in the matrix corresponds to the "weight" of a term in the document
  - zero means the term has no significance in the document or it simply doesn't exist in the document
- How to compute $w_{ij}$
  - **TF-IDF (or TF.IDF)**
  - Term frequency–inverse document frequency

$$
\begin{array}{c|cccc}
 & d_1 & d_2 & \ldots & d_n \\
\hline
t_1 & w_{11} & w_{12} & \ldots & w_{1n} \\
t_2 & w_{21} & w_{22} & \ldots & w_{2n} \\
\vdots & \vdots & \vdots & & \vdots \\
\vdots & \vdots & \vdots & & \vdots \\
t_t & w_{t1} & w_{t2} & \ldots & w_{tn}
\end{array}
$$

# TF-IDF: Term Frequency

- To compute the *term frequency*, we can use a term-document ***count matrix***

Documents

Terms

| | Email 1 | Email 2 | Email 3 | Email 4 | Email 5 | Email 6 |
|---|---|---|---|---|---|---|
| lotto | **4** | 0 | 0 | **3** | 0 | 0 |
| mr | 1 | 0 | 0 | 1 | 0 | 0 |
| bear | **2** | 0 | 0 | 0 | **4** | 0 |
| gunter | 0 | **5** | 0 | 0 | 0 | 0 |
| doggy | 0 | 0 | 0 | 0 | 0 | **5** |
| win | 0 | **2** | **3** | 0 | 0 | 0 |

# Log-Frequency Weighting

- The ***raw term frequency***, $\boldsymbol{tf_{t,d}}$, of term $\boldsymbol{t}$ in document $\boldsymbol{d}$ is defined as the number of times that $\boldsymbol{t}$ occurs in $\boldsymbol{d}$
  - However, relevance does not increase proportionally with raw term frequency
  - To this end, we can use *log-frequency weighting*

- The ***log-frequency weight***, $\boldsymbol{w_{t,d}}$, of term $\boldsymbol{t}$ in document $\boldsymbol{d}$ is:

$$\boldsymbol{w_{t,d}} = \begin{cases} \mathbf{1} + \boldsymbol{log}\left(tf_{t,d}\right) \boldsymbol{if}\ tf_{t,d} > \mathbf{0} \\ \\ \mathbf{0}\ \boldsymbol{otherwise} \end{cases}$$

# Log-Frequency Weighting: Example

- Here is our previous term-document ***count matrix***

**Documents**

| | Email 1 | Email 2 | Email 3 | Email 4 | Email 5 | Email 6 |
|---|---|---|---|---|---|---|
| **lotto** | **4** | 0 | 0 | **3** | 0 | 0 |
| **mr** | 1 | 0 | 0 | 1 | 0 | 0 |
| **bear** | **2** | 0 | 0 | 0 | **4** | 0 |
| **gunter** | 0 | **5** | 0 | 0 | 0 | 0 |
| **doggy** | 0 | 0 | 0 | 0 | 0 | **5** |
| **win** | 0 | **2** | **3** | 0 | 0 | 0 |

$$tf_{win,\ Email\ 3} = 3 > 0$$

$$tf_{win\ \ Email\ 5} = 0$$

# Log-Frequency Weighting: Example

- Here is the corresponding term-document ***log frequency matrix***

**Documents**

| | Email 1 | Email 2 | Email 3 | Email 4 | Email 5 | Email 6 |
|---|---|---|---|---|---|---|
| **lotto** | 1.602 | 0 | 0 | 1.477 | 0 | 0 |
| **mr** | 1 | 0 | 0 | 1 | 0 | 0 |
| **bear** | 1.301 | 0 | 0 | 0 | 1.602 | 0 |
| **gunter** | 0 | 0 | 0 | 0 | 1 | 0 |
| **doggy** | 0 | 1.698 | 0 | 0 | 0 | 1.698 |
| **win** | 0 | 1.301 | 1.477 | 0 | 0 | 0 |

$$w_{win,\,Email\,3} = 1 + log(tf_{win,\,Email\,3}) = 1 + log(3) = 1.477$$

$$w_{win,\,Email\,5} = 0$$

# Document frequency

- Rare terms are more informative than frequent terms
  - Recall stop words: “a”, “the”, “to”, “of”, etc., are frequent but not very informative
- Want higher weights for rare terms than for more frequent terms
- Use document frequency to capture this

# Document frequency

- **Document frequency** $\boldsymbol{df_t}$, of term $\boldsymbol{t}$
  - the number of documents in the given collection that contain $\boldsymbol{t}$
- Thus, $\boldsymbol{df_t} \leq \boldsymbol{N}$, where $\boldsymbol{N}$ is the number of documents in the collection

- $\boldsymbol{df_t}$ is an inverse measure of the informativeness of $\boldsymbol{t}$
  - *the smaller the number of documents that contain* ***t*** *the more informative* ***t*** *is*
- **Inverse document frequency**, $\boldsymbol{idf_t}$, is defined as:

$$\boldsymbol{idf_t} = \boldsymbol{log}\left(\frac{\boldsymbol{N}}{\boldsymbol{df_t}}\right)$$

If $\boldsymbol{df_t}$ = 1, $\boldsymbol{idf_t} = \boldsymbol{log(N)}$
If $\boldsymbol{df_t}$ = N, $\boldsymbol{idf_t} = \boldsymbol{0}$

# Document frequency

- **Document frequency** $\boldsymbol{df_t}$, of term $\boldsymbol{t}$
  - the number of documents in the given collection that contain $\boldsymbol{t}$
- Thus, $\boldsymbol{df_t} \leq \boldsymbol{N}$, where $\boldsymbol{N}$ is the number of documents in the collection

- $\boldsymbol{df_t}$ is an inverse measure of the informativeness of $\boldsymbol{t}$
  - *the smaller the number of documents that contain* $\boldsymbol{t}$ *the more informative* $\boldsymbol{t}$ *is*
- **Inverse document frequency**, $\boldsymbol{idf_t}$, is defined as:

$$\boldsymbol{idf_t} = \boldsymbol{log}\left(\frac{\boldsymbol{N}}{\boldsymbol{df_t}}\right)$$

**Note**: There is only one value of $\boldsymbol{idf_t}$ for each $\boldsymbol{t}$ in the collection

# TF-IDF Weighting

- A typical combined term importance indicator is *TF.IDF weighting*:

$$\boldsymbol{w_{t,d} = TF.IDF = (1 + log(}tf_{t,d}\boldsymbol{)) \times\ log\left(\frac{N}{df_t}\right)}$$

- A term occurring frequently in the document but rarely in the rest of the collection is given high weight
- **TF.IDF** increases with:
  - the number of occurrences of term ***t*** in document ***d***
  - the rarity of ***t*** in the collection

# TF-IDF: Example

- here is the respective term-document ***TF-IDF matrix***

**Documents**

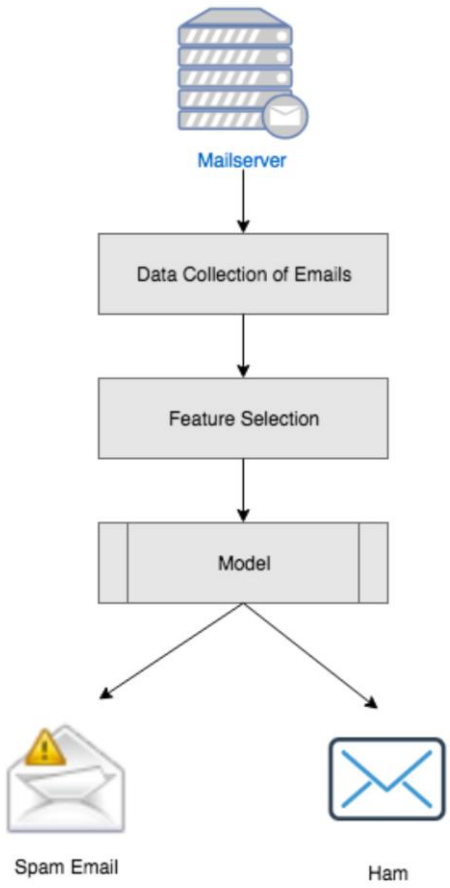
| | Email 1 | Email 2 | Email 3 | Email 4 | Email 5 | Email 6 |
|---|---|---|---|---|---|---|
| **lotto** | 0.764 | 0 | 0 | 0.704 | 0 | 0 |
| **mr** | 0.477 | 0 | 0 | 0.477 | 0 | 0 |
| **bear** | 0.620 | 0 | 0 | 0 | 0.764 | 0 |
| **gunter** | 0 | 0 | 0 | 0 | 0.778 | 0 |
| **doggy** | 0 | 0.810 | 0 | 0 | 0 | 0.810 |
| **win** | 0 | 0.620 | 0.704 | 0 | 0 | 0 |

$$w_{win,\,Email\,3} = [1 + log(tf_{win,\,Email\,3})] \times log\left(\frac{N}{df_{win}}\right) = 1.477 \times log\left(\frac{6}{2}\right) = 0.704$$

# Spam Detection

- Task: separate spam emails from a set of non-spam emails
  - Given a large collection of example emails, each labeled “spam” or “ham”
  - Learn to predict labels of new, future emails
  - Binary classification.

- How: use your favorite classifier(s) to distinguish between spam and ham emails.



# Summary

- Spam and phishing
- Spam Filtering Techniques
- Pre-processing Text in Email Messages
  - bag-of-words
  - n-gram
  - TF-IDF

# References


- *Email Spam Filtering, Computer Security Seminar, N.Muthiyalu Jothir*
- *Special Topics: Adversarial Machine Learning, Dr. Alex Vakanski, University of Idaho*
- *Introduction to Information Retrieval, Christopher D. Manning et al, Cambridge University Press. 2008.*
- *Machine Learning and Security Protecting Systems with Data and Algorithms, Clarence Chio, David Freeman*